THRIFTCIRCLE API DOCUMENTATION

Digital **Àjọ** / **Èsúsú** Savings Management Platform — API v1.0

AUTHOR
Precious Omolaolu

AUTHENTICATION
Bearer JWT Access Token

BASE URL
http://localhost:3000/api

1.0 ENDPOINT OVERVIEW

This documentation details the API specifications for ThriftCircle, focusing on Group Membership Administration, Dynamic Payout Queue Management, and the Payout Phase Lifecycle. Organizers can manage pending member join requests, reorder payout delivery sequences, track automated contribution aggregations, approve payout disbursements, and monitor group state progression across active, payout, and completed phases.

2.0 AUTHENTICATION & BASE CONFIGURATION

All requests require a valid JWT access token supplied via the HTTP Authorization header:

```
Authorization: Bearer <your_jwt_access_token>
```

Base URLs:

- **Development:** `http://localhost:3000/api`
- **Production:** `https://api.thriftcircle.com/api`

3.0 COMPLETE API ROUTE REGISTRY

POST	`/api/auth/signup`	Register a new user account
POST	`/api/auth/login`	Authenticate user & return tokens
POST	`/api/auth/forgot-password`	Request password reset token
POST	`/api/auth/reset-password`	Complete password reset
POST	`/api/auth/refresh`	Refresh access token
POST	`/api/auth/logout`	Invalidate active session
GET	`/api/health`	System health check
POST	`/api/groups`	Create a thrift group
POST	`/api/groups/join`	Join an existing group

GET	/api/groups/mine	Retrieve user's groups
GET	/api/groups/:groupId	Get specific group details
GET	/api/groups/:groupId/pending	List pending member requests
PATCH	/api/groups/:groupId/members/:userId/approve	Approve pending member request
PATCH	/api/groups/:groupId/members/:userId/reject	Reject pending member request
GET	/api/groups/:groupId/contributions	List group contributions
POST	/api/groups/:groupId/contributions	Record a contribution
GET	/api/contributions/:contributionId	Get contribution details
POST	/api/contributions/:contributionId/evidence	Upload payment receipt image
PATCH	/api/evidence/:evidenceId/review	Review payment evidence
POST	/api/issues/report	Report group or platform issue
GET	/api/groups/:groupId/issues	List group issues
PATCH	/api/issues/:id/resolve	Resolve reported issue
GET	/api/groups/:groupId/cycles/current	Get current active cycle
POST	/api/groups/:groupId/payouts	Trigger Payout Phase (Organizer)
GET	/api/groups/:groupId/payouts	List group payouts
PUT	/api/groups/:groupId/payouts/order	Reorder payout recipients
PATCH	/api/payouts/:payoutId/approve	Approve payout disbursement
GET	/api/groups/:groupId/dashboard/organizer	Organizer analytical dashboard
GET	/api/groups/:groupId/dashboard/member	Member summary dashboard
GET	/api/groups/:groupId/outstanding	View outstanding balances
GET	/api/groups/:groupId/progress	View cycle/group progress metrics
GET	/api/notifications	Fetch user notifications
PATCH	/api/notifications/read-all	Mark all notifications as read
PATCH	/api/notifications/:notificationId/read	Mark single notification as read

4.0 CORE ENDPOINT SPECIFICATIONS & PAYLOADS

PATCH /api/groups/:groupId/members/:userId/approve

Allows group organizers to accept a pending membership request.

Request Body (Application/Json):

```
{
  "status": "APPROVED",
  "reason": "Verified member background check."
}
```

PUT /api/groups/:groupId/payouts/order

Enables the organizer to update the receiving sequence of payout slots across group members at any time. Members who have already received an approved payout cannot be reordered (returns 409 Conflict).

Request Body (Application/Json):

```
{
  "payoutOrder": [
    { "position": 1, "memberId": "65e987654321fedcba654321" },
    { "position": 2, "memberId": "65e987654321fedcba654322" },
    { "position": 3, "memberId": "65e987654321fedcba654323" }
  ]
}
```

POST /api/groups/:groupId/payouts

The organizer manually triggers the Payout Phase. This action simultaneously creates the payout as `pending_approval` and shifts the cycle from `ACTIVE` to `PAYOUT_PHASE`. Requires at least one confirmed contribution.

PATCH /api/payouts/:payoutId/approve

Organizers approve the pending payout, setting it to completed and finalizing the cycle once all active member contributions are confirmed.

Request Body (Application/Json):

```
{
  "transactionNotes": "Payout disbursed following receipt validation for cycle 2."
}
```

5.0 ERROR CODES

400 Bad Request	`INVALID_STATUS`	Status must be explicitly passed as APPROVED or REJECTED.
403 Forbidden	`ORGANIZER_ROLE_REQUIRED`	

		Non-organizer account attempted restricted administrative action.
409 Conflict	`UNCONFIRMED_OR_LOCKED`	Payout triggered without confirmed contributions or reordering locked for paid member.
422 Unprocessable	`INVALID_CYCLE_PHASE`	Action attempted in an incompatible cycle state.